FashionStore repository guide

Nov 6, 2026 · @The King

Everything is written and every suite that can run without AWS is green; nothing has touched a real AWS account yet. "A" is the platform builder, "B" is the data and app builder.

How the pieces fit

A shopper only ever talks to CloudFront; everything else is private.

1. The browser loads the React site from a private S3 bucket through CloudFront (OAC).
2. Sign-in goes to the Cognito hosted UI (authorization code + PKCE) and returns an access token.
3. `/api/*` goes from CloudFront through a VPC origin to the internal ALB, which sends 90% to store-api v1 pods and 10% to v2 pods on EKS.
4. `POST /api/checkouts` saves the checkout as PENDING in RDS MySQL and sends it to the SQS `checkout-requests` queue.
5. The process-checkout Lambda decrements stock only if enough is left, calls payment-api, writes the order and publishes the outcome to SNS `checkout-status`.
6. The notifier Lambda emails the shopper once via SES. The storefront polls the status until it is CONFIRMED or REJECTED.
7. Every night the report Lambda writes sales and stock CSVs to S3 for Athena.

Safety nets: a failed message is retried and lands in the DLQ after 5 tries; the store-api reconciler re-sends checkouts that never reached SQS and re-publishes outcomes that were never published.

Top level, docs, contracts and database

Path	What it does
`README.md`	Front page: status table, offline test commands, the deploy runbook

Path	What it does
`Makefile`	Every command the team runs; `make help` lists them. Offline: `test`, `tf-test`, `py-test`, `api-test`, `web-test`, `e2e`. AWS: `bootstrap`, `deploy-demo`, `k8s-config`, `addons`, `db-migrate-remote`, `push-api`, `k8s-apply`, `deploy-storefront`, `smoke`, `verify-private`, `live-test`, `e2e-live`, `load-*`, `destroy-demo`
`.gitignore`	Keeps state, `terraform.tfvars`, `backend.hcl`, build output and `node_modules` out of git
`.github/workflows/ci.yml`	On every PR: Terraform fmt/validate/test, Lambda pytest on MySQL, Kustomize render, store-api tests and Docker build, storefront tests and build, Playwright e2e, k6 script parse
`.github/workflows/terraform-plan.yml`	On PRs touching `infra/` or `functions/`: assumes the read-only GitHub OIDC role and runs `terraform plan`
`docs/implementation-plan.md`	Locked decisions, who does what, the 5-day schedule with exit criteria, the teardown checklist, risks
`docs/repository-guide.md`	This guide, as a file in the repo
`contracts/openapi.yaml`	Every `/api/*` endpoint. store-api implements it, the storefront calls it
`contracts/events/checkout-request.schema.json`	The SQS message store-api sends to process-checkout
`contracts/events/checkout-status.schema.json`	The SNS outcome message (CONFIRMED or REJECTED + reason)
`contracts/examples/*.json`	One valid example per message; tests validate them against the schemas

Path	What it does
`contracts/README.md`	Who produces and consumes what; at-least-once delivery, dedupe on `checkout_id`, never trust prices in a message
`database/migrations/V001__catalog_and_cart.sql`	`products`, `inventory` (CHECK qty >= 0 as a backstop), `carts`, `cart_items`
`database/migrations/V002__checkout_and_orders.sql`	`checkouts` (PENDING rows, `enqueued_at` for the reconciler), `orders` (UNIQUE `checkout_id`), `order_items`
`database/migrations/V003__dispatch_and_notification.sql`	`notification_log` (UNIQUE `checkout_id`, lease column `claimed_until`) and `reconciliation_log`
`database/seeds/*.sql`	10 demo products; `FLASH-001` has 50 units for the flash sale. Re-runnable
`database/queries/verify-*.sql`	Evidence queries: no negative stock, one order and one email per checkout, report equals orders
`database/README.md`	Migrations run as a Kubernetes Job because RDS sits in isolated subnets

functions (four Lambdas, Python 3.12 arm64)

Path	What it does
`common/fashionstore_common/db.py`	Opens MySQL with the password from Secrets Manager; reconnects after a rotation
`common/fashionstore_common/testing.py`	Test helper: builds a throwaway database from `database/migrations`
`process-checkout/handler.py`	SQS entry point; reports failed messages one by one so only those are retried
`process-checkout/service.py`	The checkout rules: validate, price from the DB, pay, place or reject, publish

Path	What it does
`process-checkout/repository.py`	The SQL: UNIQUE order insert (idempotency) and `UPDATE inventory ... WHERE qty >= ?` in SKU order (no oversell, no deadlock)
`process-checkout/tests/`	31 tests, including 120 buyers racing for 50 units (exactly 50 confirmed) and duplicate deliveries
`payment-api/handler.py`	Fake payment provider: same answer for the same checkout; declines emails containing `+decline` (12 tests)
`notifier/handler.py`, `repository.py`	SNS → claim a `notification_log` row with a lease → SES send → mark SENT; one email per checkout (10 tests)
`notifier/templates/*.html`	The confirmation and rejection emails
`report/handler.py`, `queries/*.sql`	Daily sales and stock CSVs into S3, partitioned for Athena (3 tests)
`*/requirements.txt`	Runtime dependencies packed into each zip
`requirements-dev.txt`	pytest, boto3, moto, jsonschema, PyMySQL for local tests
`README.md`	What each function guarantees and how to test it

apps/store-api (Spring Boot 3.5, Java 21)

Paths under `src/main/java/com/fashionstore/` are shortened to the package name.

Path	What it does
`pom.xml`, `mvnw`, `.mvn/`	Maven build; the wrapper means no Maven install is needed
`Dockerfile`, `.dockerignore`	Two-stage image: build the jar, run it on a JRE as a non-root user
`StoreApiApplication.java`	Starts the app in UTC with scheduling on

Path	What it does
`configuration/AppProperties.java`	All `fashionstore.*` settings in one typed record
`configuration/AwsConfig.java`	SQS and SNS clients
`configuration/DataSourceConfig.java`	Reads the DB password from Secrets Manager when `DB_SECRET_ARN` is set
`configuration/SecurityConfig.java`	Catalogue public; cart and checkout need `checkout.write`; orders need `orders.read`; everything else denied
`auth/CognitoAccessTokenValidator.java`	Accepts only access tokens issued to our client (ID tokens refused)
`auth/UserEmailResolver.java`, `CurrentUser.java`	Who is calling, and their email
`catalog/*`	`GET /api/products` and `/api/products/{sku}` with live stock
`cart/*`	`GET/PUT /api/cart`, stored in MySQL
`checkout/CheckoutService.java`	Save PENDING and clear the cart in one transaction, then send to SQS. Same Idempotency-Key = same checkout; another shopper's key = 409
`checkout/CheckoutController.java`, `CheckoutRepository.java`, `CheckoutStatus.java`	The endpoints and SQL for checkouts
`orders/*`	`GET /api/orders` and `/api/orders/{id}`, only your own (others get 404)
`messaging/Reconciler.java`	Every 15 s: re-send checkouts that never reached SQS, re-publish outcomes never published. `SKIP LOCKED` so 3 pods never collide
`messaging/Sqs*`, `Sns*`, `InMemoryMessaging.java`	Real AWS senders, plus in-memory fakes for local runs and tests
`common/*`	Error JSON with `correlation_id`, the `X-Correlation-Id` filter, `/api/version` (v1 or v2)

Path	What it does
`resources/application.yml`	Maps environment variables from the Kubernetes ConfigMap to settings; readiness checks the DB
`resources/application-local.yml`	Local profile: no AWS, messages kept in memory
`src/test/java/...`	23 integration tests on a real MySQL container: security, cart, idempotency, SQS outage, reconciler, ownership
`README.md`	Package map and how a checkout stays correct

apps/storefront (React 19, Vite)

Path	What it does
`package.json`, `package-lock.json`	Dependencies and scripts: `dev`, `build`, `test`, `typecheck`
`index.html`, `vite.config.ts`, `tsconfig.json`	Vite entry, dev proxy of `/api` to :8080, test setup
`public/config.json`	Runtime settings; the deploy script overwrites it with the real Cognito values, so one build works anywhere
`public/images/*.svg`	Placeholder product pictures
`.env.example`	Explains why there are no build-time env vars
`src/main.tsx`, `src/App.tsx`	Loads `config.json`, then the routes
`src/auth/pkce.ts`	Cognito sign-in: PKCE challenge, state check, code exchange
`src/auth/AuthContext.tsx`	Keeps tokens in sessionStorage; `useAuth()`
`src/api/client.ts`, `index.ts`, `ApiContext.tsx`	Typed calls for every endpoint, bearer token attached
`src/hooks/useCart.ts`	Server-side cart

Path	What it does
`src/hooks/useCheckoutPolling.ts`	Polls a checkout with backoff (0.5 s to 4 s, gives up at 120 s)
`src/pages/*`	Products, details, bag, checkout, status, orders, order details, sign-in callback
`src/pages/checkoutKey.ts`	One Idempotency-Key per checkout attempt, reused on retry
`src/components/*`	Navigation, product card, cart line, checkout status panel
`src/styles/app.css`, `src/types/index.ts`	Styling and shared types
`tests/*`	15 vitest tests: RFC 7636 PKCE vector, API client, polling, retry keeps the same key, components
`README.md`	Layout and local development

infra/terraform (all AWS resources)

Each module has `main.tf`, `variables.tf`, `outputs.tf`, `versions.tf` and a `tests/*.tftest.hcl` that checks its locked decisions against a mocked AWS provider.

Path	What it does
`bootstrap/`	Run once, local state: state bucket, GitHub OIDC plan and apply roles, $100 budget with 50 / 80 / forecast-100 alerts
`environments/demo/main.tf`	Wires all modules into the demo stack
`environments/demo/variables.tf`, `terraform.tfvars.example`	Inputs: owner, expiry date, SES sender and recipients, EKS admin ARNs, v1/v2 weights
`environments/demo/outputs.tf`	URLs, ARNs and `app_config` that the scripts read
`environments/demo/backend.hcl.example`, `providers.tf`, `versions.tf`	S3 backend with lockfile, AWS provider with default tags
`environments/demo/tests/stack.tftest.hcl`	Whole-stack composition test

Path	What it does
`modules/network`	VPC, public/private/isolated subnets in 2 AZs, one NAT, S3 gateway endpoint, security groups
`modules/registry`	ECR repo for store-api (immutable tags, scan on push)
`modules/database`	RDS MySQL 8.0 db.t4g.micro, password managed in Secrets Manager
`modules/messaging`	SQS checkout queue + DLQ, notifier-failures queue, SNS topics
`modules/analytics`	Reports bucket, Athena workgroup, Glue tables
`modules/functions`	The four Lambdas, their roles, triggers, schedule, SES identities
`modules/eks`	EKS 1.33, t3.medium node group 1/2/3, IRSA roles
`modules/delivery`	Internal ALB, v1/v2 target groups 90/10, S3 + OAC, CloudFront VPC origin, `spa-rewrite.js`
`modules/identity`	Cognito pool, classic hosted UI, PKCE client, API scopes
`modules/observability`	11 alarms to email, one dashboard
`modules/private-bucket`	Helper for private, encrypted, TLS-only buckets
`tests/mocks/aws.tfmock.hcl`	Realistic fake ARNs and ids shared by all tests
`README.md`	Module table, network map, first-run steps

platform and scripts

Path	What it does
`platform/kubernetes/base/namespace.yaml`, `service-account.yaml`	Namespace and the IRSA service account for store-api
`platform/kubernetes/base/deployment-v1.yaml`, `deployment-v2.yaml`	store-api v1 (2 pods) and v2 (1 pod), probes on `/actuator/health/*`
`platform/kubernetes/base/service.yaml`, `target-group-binding.yaml`	Services, and bindings that register pod IPs in the Terraform-made target groups
`platform/kubernetes/base/hpa-v1.yaml`	Autoscaling for v1, plus a disruption budget
`platform/kubernetes/base/configmap.yaml`	Defaults shared by both versions
`platform/kubernetes/overlays/demo/*`	Fills in real values from `terraform output` (`generated/*.env`, written by `make k8s-config`)
`platform/kubernetes/jobs/database-migration.yaml`	Flyway Job that migrates RDS from inside the cluster
`platform/kubernetes/jobs/seed-demo.yaml`	Job that loads the seeds
`platform/addons/*/values.yaml`	Helm settings for metrics-server, Cluster Autoscaler, AWS Load Balancer Controller
`scripts/package-functions.sh`	Builds `build/functions/*.zip` (code + common + dependencies), byte-for-byte repeatable
`scripts/render-k8s-config.sh`	Writes the overlay's `generated/*.env` from `terraform output`
`scripts/select-kinds.py`	Filters rendered YAML by kind (used before the migration Job)
`scripts/build-push-api.sh`	Builds the store-api image for amd64 and pushes it to ECR
`scripts/deploy-storefront.sh`	Builds the site, writes `config.json`, uploads to S3 with cache headers, invalidates CloudFront

Path	What it does
`scripts/create-test-users.sh`	Creates confirmed Cognito test shoppers

tests

Only `e2e` runs offline; everything else here needs the deployed stack and skips cleanly without it.

Path	What it does
`README.md`	Every test, where it lives, how to run it
`infrastructure/smoke-test.sh`	Live: one checkout straight into SQS, expects CONFIRMED and an empty DLQ
`infrastructure/verify-private-resources.sh`	Live: ALB internal, RDS private, S3 403, CloudFront 200 and HTTPS redirect
`e2e/playwright.config.ts`, `package.json`	Playwright setup; mock mode by default, live mode when `E2E_BASE_URL` is set
`e2e/specs/shopper-journey.spec.ts`	Browse, sign in, add to bag, checkout, confirmed, orders; a retry keeps the same key
`e2e/specs/rejected-checkout.spec.ts`	Declined payment shows a clear message; signed-out users are asked to sign in
`e2e/support/mock-backend.ts`	Fake Cognito and API inside the browser for offline runs
`e2e/support/session.ts`, `env.ts`	Sign-in steps and test users for both modes
`e2e/tools/get-token.mjs`	Gets real tokens through the hosted UI, for the live pytest and k6 tests
`conftest.py`, `pytest.ini`, `requirements-live.txt`	Shared live-test setup; skips unless `FS_BASE_URL` is set
`security/test_authentication.py`	401 with no token, a forged token or an ID token

Path	What it does
`security/test_ownership.py`	Shopper B can't read or reuse shopper A's checkout or order
`security/test_origin_access.py`	Only CloudFront is reachable; SPA deep links work; API 404s stay JSON
`correctness/test_checkout_live.py`	Confirm, stock drops, idempotent retry, empty cart, decline path, 90/10 split
`failure/test_dlq_redrive.py`	A message moved back from the DLQ becomes an order
`failure/test_worker_retry.py`	Payment outage recovers through SQS retry (chaos); a poison message reaches the DLQ (slow)
`failure/test_dispatch_recovery.py`	store-api can't reach SQS, the reconciler sends it later (chaos)
`failure/test_notification_recovery.py`	Notifier outage, then exactly one email (chaos)
`load/browse.js`, `checkout.js`, `flash-sale.js`	k6 load tests with p95 thresholds; the flash sale checks for oversell
`load/lib/common.js`, `profiles/*.json`, `make-tokens.sh`	Shared helpers, smoke and demo sizes, tokens for N load-test users

Chaos tests change the stack on purpose (Lambda concurrency 0, a bogus queue URL) and restore it afterwards; they run only with `FS_ALLOW_CHAOS=1`, one at a time.

What is done

All counts below come from one re-run of every suite on a clean copy of the delivered files.

Area	Evidence
Terraform: bootstrap, 10 modules, demo stack	31 `terraform test` runs green on a mocked provider; `fmt` and `validate` clean
Database schema and seeds	Flyway V001–V003 migrate cleanly on MySQL 8.0

Area	Evidence
Four Lambdas	56 pytest tests green on real MySQL 8, including the oversell race and duplicate deliveries
store-api	23 integration tests green on Testcontainers MySQL; the jar starts locally and serves the catalogue
storefront	15 unit tests, typecheck and production build green
Browser journeys	4 Playwright tests green on the mocked backend, 12 of 12 on a 3x repeat
Live tests and load scripts	Written; all 35 live tests skip cleanly offline; the public and no-token auth checks passed against a local store-api
Kubernetes, Helm, deploy scripts, CI	Written; the overlay renders with `kubectl kustomize`; Lambda zips build

One design fix along the way: Cognito `USER_PASSWORD_AUTH` was removed. Its tokens never carry the `fashionstore-api/*` scopes the API requires, so every scripted test would have got 403. Scripts now sign in through the hosted UI with PKCE.

What still needs to be done

Everything left needs the AWS account, so it is the 5-day window's work, in this order.

- [ ] **A:** Create the GitHub repo and push the code; after the next step, set the `AWS_PLAN_ROLE_ARN` and `TF_STATE_BUCKET` repo variables. No repo exists yet.
- [ ] **A:** `make bootstrap`, then fill `environments/demo/terraform.tfvars` (owner, `expires_on`, SES sender and recipients, your IAM ARN in `eks_admin_principal_arns`, your IP in `eks_public_access_cidrs`).
- [ ] **A:** `make init-demo deploy-demo`. Never applied yet, so expect a few fixes on the first real apply.
- [ ] **B:** Click the SES verification emails for the sender and every recipient.
- [ ] **A:** `make k8s-config addons db-migrate-remote db-seed-remote`, then `make smoke`.
- [ ] **B builds, A applies:** `make push-api TAG=v1` and `TAG=v2`, then `make k8s-config V1_TAG=v1 V2_TAG=v2 k8s-apply`. The Docker image has never been built, because Docker Hub and ECR Public were blocked here.

- [ ] **B:** `make deploy-storefront verify-private`. If `/api/*` gives 502 or 504, add the CloudFront VPC-origin security group to `alb_extra_source_security_group_ids`.
- [ ] **B:** `make test-users`, then `make live-test e2e-live`. This is the first real run of `get-token.mjs`; the hosted-UI field names are an educated guess.
- [ ] **A + B:** Chaos and slow tests with `FS_ALLOW_CHAOS=1 FS_RUN_SLOW=1 make live-test`, one at a time.
- [ ] **A + B:** Load: `make load-browse load-checkout load-flash PROFILE=smoke`, then `PROFILE=demo`. k6 has not been run yet; reseed FLASH-001 before each flash sale.
- [ ] **A:** Pilot: set the weights to 50/50, then 0/100 and back, `terraform apply` each time, and record the target-group metrics.
- [ ] **B:** Report: invoke `report` with `{"date": "today"}` and run the Athena named queries.
- [ ] **A + B:** Save evidence: screenshots, k6 summaries, CloudWatch graphs, Cost Explorer, `terraform state list`. No `evidence/` folder exists yet.
- [ ] **A:** Day 5: run the teardown checklist in the plan, then check the bill.

Known gaps that are not built: there is no CD workflow (deploys are `make` commands run by hand), and the Kubernetes YAML has been rendered but not schema-checked with kubeconform.